

ATUALIZAÇÃO ARQUITETURAL: AMBIENTE KVM LINUX

OTIMIZAÇÕES CRÍTICAS DE I/O, REDE E PROCESSAMENTO PARA O VMM

Documento: Adendo Técnico ao Guia de Configuração de Testes

Data: Maio de 2026

Ambiente: Laptop Ryzen 5 3500U | 16GB RAM | SSD NVMe

Alvo: Zorin OS 18.1 (Base Ubuntu 24.04 LTS) / VMM

1. Diagnóstico do Cenário e Justificativa Tecnológica

O ecossistema de testes implementado baseia-se numa arquitetura de **cache em três níveis** altamente eficiente: o proxy local APT-Cacher-NG, o repositório de ficheiros estáticos via NFS (em /tmp/cache) e o pseudo-cache Flatpak/ostree (em /mnt/.ostree/repo/). Esta abordagem otimiza drasticamente o tráfego de rede externa, atingindo taxas de eficiência de cache superiores a 95%.

Contudo, sob testes intensivos de concorrência ou reconfiguração de múltiplas estações de trabalho simultâneas, o hipervisor padrão do *Virtual Machine Manager (VMM / libvirt)* gera contenção severa de I/O. A centralização dos dados no SSD NVMe e as requisições assíncronas do ecossistema de cache demandam que a camada de emulação gráfica, de rede e de armazenamento do QEMU/KVM seja atualizada para os novos padrões de kernel de 2026.

2. Otimização de I/O de Armazenamento (Passos 16 e 17 do Guia)

A configuração original utiliza o controlador VirtIO SCSI com diretivas básicas. Para extrair a largura de banda nativa do SSD NVMe corporativo sem sobrecarregar a CPU do host, as seguintes evoluções devem ser aplicadas diretamente no XML do domínio via comando `virsh edit` ou habilitando a edição de XML nas preferências do VMM:

A. Implementação de Multi-Queue no VirtIO-SCSI

Por omissão, o driver aloca apenas uma fila de submissão de comandos. Ao ativar o suporte a múltiplas filas (Multi-Queue), criamos um canal de I/O independente e uma interrupção dedicada para cada vCPU da máquina virtual, eliminando os bloqueios de concorrência internos do hipervisor.

B. Migração do Motor Assíncrono para io_uring

O subsistema do kernel Linux `io_uring` substitui com vantagens extremas os antigos modelos baseados em `threads` ou `native` (AIO legada). Ele permite o envio e recolha de chamadas de sistema de I/O sem a necessidade de constantes mudanças de contexto (`context switching`), reduzindo a latência de escrita no cache NFS.

Como atualizar no XML: Substitua a definição do controlador e do disco principal pelas diretivas abaixo:

```
<!-- Ativação do Multi-Queue no controlador SCSI -->
<controller type='scsi' index='0' model='virtio-scsi'>
  <driver queues='4' />
</controller>

<!-- Configuração do disco QCOW2 com motor io_uring -->
<disk type='file' device='disk'>
  <driver name='qemu' type='qcow2' cache='none' io='io_uring' discard='unmap' />
  <source file='/var/lib/libvirt/images/zorin-target.qcow2' />
  <target dev='sda' bus='scsi' />
</!-- Adicione logo abaixo da tag de fecho de </vcpu> -->
<iothreads>1</iothreads>

<!-- No elemento <disk>, associe o disco à thread criada -->
<driver name='qemu' type='qcow2' cache='none' io='io_uring' iothread='1' discard='unmap' />
```

4. Paralelização da Camada de Rede (Otimização APT-Cacher-NG)

A placa de rede virtual padrão trafega dados por um único canal lógico. Como a máquina de testes comunica constantemente com o host para consumir o cache do proxy HTTP e do sistema de ficheiros NFS, a pilha de rede torna-se um gargalo de software.

A ativação do suporte Multi-Queue no driver kernel host `vhost_net` permite paralelizar o processamento de pacotes pelos núcleos disponíveis do processador AMD Ryzen.

```
<!-- Modificação da interface de rede no XML -->
<interface type='network'>
  <source network='default' />
  <model type='virtio' />
  <driver name='vhost' queues='4' />
</interface>
```

Nota de Administração de Sistemas: Se o sistema convidado (Zorin OS) não ativar as filas adicionais automaticamente, adicione o seguinte comando pós-boot ou no script de customização da VM:

```
sudo ethtool -L enp1s0 combined 4 (ajustando o nome da interface conforme detetado).
```

5. Ajuste Fino da Topologia de CPU para AMD Ryzen 5 3500U (Passo 11)

O processador Ryzen 5 3500U possui arquitetura Zen+ com 4 Cores e 8 Threads (SMT ativo), compartilhando um complexo de cache L3 unificado. Configurações genéricas de vCPU ocultam a topologia real, fazendo com que o agendador do Linux guest tome decisões ineficientes de migração de threads entre núcleos virtuais.

Ao expor diretamente a arquitetura via `host-passthrough` e desenhar uma topologia explícita que reflete fielmente o SMT (Hyper-Threading) nativo, o desempenho de compilação e execução de scripts melhora sensivelmente.

```
<!-- Definição ideal de CPU para alocação de 4 vCPUs no Ryzen 3500U -->  
<cpu mode='host-passthrough' check='none'>  
  <topology sockets='1' dies='1' cores='2' threads='2' />  
</cpu>
```

6. Modernização Gráfica e de Áudio (Wayland e PipeWire)

Considerando as bases modernas do Zorin OS 18.1 (alinhadas com o ciclo de vida tecnológico de 2026), o ecossistema gráfico migrou prioritariamente para o servidor gráfico Wayland e o gerenciamento de som para o PipeWire.

- **Display Gráfico via D-Bus:** Para estações onde o sistema hospedeiro também executa Wayland, substitua o clássico servidor Spice pelo canal dbus combinado com renderização OpenGL nativa (`<gl enable='yes' />`). Isto resulta em aceleração 3D sem perdas de frames na renderização da interface desktop.
- **Subsistema de Áudio PipeWire Nativo:** Evite emular controladores legados como AC97 ou ICH9. As versões recentes do VMM/QEMU permitem mapear a saída diretamente para o barramento de áudio do host, minimizando distorções e latências em chamadas de voz ou testes multimídia.

Fim do Documento Técnico de Atualização • Preparado para o Comitê de Soberania Digital e Gestão de TI